

Hardening a RHEL Server

1. Getting the server ready

I registered the VM directly with Red Hat instead of using a local ISO repo, since that's closer to how a real production server would be set up. I checked the disk layout with `lsblk` and `blkid` — it's a single NVMe disk with LVM for root and swap. I didn't have a spare disk to build a separate data volume, and I wasn't confident enough with LVM yet to build one on the existing volume group, so I left that out on purpose rather than faking it. That's an honest gap, not a shortcut.

2. Setting up users properly

Instead of just making random test accounts, I built a small team structure: `sysadmins`, `devteam`, and `auditors` groups, with users mapped to each. I applied a real password aging policy (90-day expiry, 7-day warning) and forced one account to reset its password on next login just to see the mechanism actually work.

For a decommissioned test account, I used two separate lockdown methods at once — locking the password and also setting the shell to `nologin`. It felt slightly redundant at first, but they actually block two different things, so using both is the right call, not overkill.

I also set up two shared folders to demonstrate permission concepts I'd only really understood in theory before:

- SGID on `/srv/devteam` — anything created inside automatically belongs to the `devteam` group.
- Sticky bit on `/srv/shared-drop` — anyone can add files, but nobody can delete someone else's. I tested this by having one user try to delete another's file, and it correctly failed.

3. Locking down SSH

I generated an SSH key pair, copied the public key to the server, and only after confirming the key actually worked did I disable password logins in `sshd_config`. That order matters — turning off password auth before verifying the key would have locked me out.

Along the way I actually got locked out of a smaller thing: I'd force-expired a password in Phase 2, which meant the automated `ssh-copy-id` step couldn't complete because it needs an interactive prompt to handle an expired password, and it doesn't have one. I had to log in manually first to reset the password properly. It was a good reminder that the policy I'd set up was actually working, not just sitting there as a config line.

I also set a proper hostname (`servera.rhelfproject.local`) and along the way noticed RHEL used Polkit rather than plain `sudo` to authorize that change — a small detail, but a useful one to know when troubleshooting permissions on a modern system.

4. Logging and keeping time honest

This phase had the most interesting troubleshooting moment of the whole project. I sent a test log message and it didn't show up where I expected. Instead of assuming something was broken, I checked `rsyslog`'s config and found that `local7` messages are deliberately routed to `/var/log/boot.log`, not the general log file — that's just how RHEL is set up by default. The message was there the whole time, just in the right file, not the one I assumed.

I also switched `journald` over to persistent storage instead of its default RAM-only mode, so logs survive a reboot. Once that was working, I pulled up the SSH login history and could see the exact moment my key-based login from Phase 3 succeeded, preserved permanently in the log. That was a nice moment where two separate phases of the project actually connected to prove each other worked.

Finally, I got `chrony` running for time sync — small thing, but if a server's clock drifts, log timestamps stop lining up across systems, which makes troubleshooting anything a lot harder.

5. Backups and staying patched

I backed up `/etc` using `tar` with `xz` compression — about 30MB compressed down to under 4MB. I checked the archive's contents without extracting it, just to confirm it was actually intact.

For patching, I treated it as a security task rather than routine maintenance — running `dnf update --security` specifically, and looking at `dnf`'s transaction history, which works as a built-in audit trail and can even be rolled back if an update causes a problem.

6. The actual security audit

This was the phase that tied everything together. I ran a system-wide search for SUID and SGID binaries, checked for world-writable files, and checked for any accounts with blank passwords.

Almost everything came back as expected system files — but one result stood out: an SGID script sitting at `/projects/phoenix/shared/sync.sh`, in a completely non-standard location. That's exactly the kind of thing an audit is supposed to catch. In a real environment I'd follow up on who owns it, what it actually

does, and whether the SGID bit was even meant to be there. Finding that felt like the moment the project stopped being a checklist and started being an actual audit.

Last, I set up Cockpit (the web-based admin dashboard) and made sure it was only reachable through a specific firewall rule, rather than just opening the port to everything.